سوال مصاحبه : چه دیزاین پترن هایی توی پروژه ازشون استفاده کردی ؟

دیزاین پترن هایی که در پروژه دات نت ام وی سی هست.

✅ 1. MVC Pattern

```
// Model
public class Product {
    public int Id { get; set; }
    public string Name { get; set; }
}

// Controller
public class ProductController : Controller {
    public ActionResult Details(int id) {
        var product = new Product { Id = id, Name = "Book" };
        return View(product);
    }
}

// View (Details.cshtml)
@model Product
<h2>@Model.Name</h2>
```

✅ 2. Repository Pattern

```
// Interface
public interface IProductRepository {
    Product GetById(int id);
}

// Implementation
public class ProductRepository : IProductRepository {
    private readonly AppDbContext _context;
    public ProductRepository(AppDbContext context) => _context = context;

    public Product GetById(int id) => _context.Products.Find(id);
}

// استفاده در Controller
public class ProductController : Controller {
    private readonly IProductRepository _repo;
    public ProductController(IProductRepository repo) => _repo = repo;

    public ActionResult Details(int id) {
        var product = _repo.GetById(id);
        return View(product);
    }}
```

✅ 3. Unit of Work Pattern

```
public interface IUnitOfWork {
    IProductRepository Products { get; }
    void Complete();
}

public class UnitOfWork : IUnitOfWork {
    private readonly AppDbContext _context;
    public IProductRepository Products { get; }

    public UnitOfWork(AppDbContext context) {
        _context = context;
        Products = new ProductRepository(_context);
    }

    public void Complete() => _context.SaveChanges();
}
```

✅ 4. Dependency Injection (در Startup.cs)

```
services.AddScoped<IProductRepository, ProductRepository>();
services.AddScoped<IUnitOfWork, UnitOfWork>();
```

✅ 5. Factory Pattern

```
public interface IMessage {
    string GetMessage();
}

public class EmailMessage : IMessage {
    public string GetMessage() => "Email Sent!";
}

public class SmsMessage : IMessage {
    public string GetMessage() => "SMS Sent!";
}

public class MessageFactory {
    public static IMessage Create(string type) =>
        type == "email" ? new EmailMessage() : new SmsMessage();
}

// استفاده در Controller
var message = MessageFactory.Create("email");
var text = message.GetMessage(); // "Email Sent!"
```

✅ 6. Singleton Pattern

```
public class AppSettings {
    private static AppSettings _instance;
    public string SiteName { get; set; } = "MySite";

    private AppSettings() {}

    public static AppSettings Instance => _instance ??= new AppSettings();
}

// استفاده:
string name = AppSettings.Instance.SiteName;
```

✅ 7. Strategy Pattern

```
public interface IPaymentStrategy {
    string Pay();
}

public class PaypalPayment : IPaymentStrategy {
    public string Pay() => "Paid with PayPal";
}

public class CreditCardPayment : IPaymentStrategy {
    public string Pay() => "Paid with Credit Card";
}

public class PaymentContext {
    private readonly IPaymentStrategy _strategy;
    public PaymentContext(IPaymentStrategy strategy) => _strategy = strategy;

    public string ExecutePayment() => _strategy.Pay();
}

// استفاده
var context = new PaymentContext(new PaypalPayment());
string result = context.ExecutePayment(); // "Paid with PayPal"
```

✅ 8. Decorator Pattern

```
public interface INotifier {
    void Send(string message);
}

public class EmailNotifier : INotifier {
```

```
    public void Send(string message) => Console.WriteLine("Email: " + message);
}

public class NotifierDecorator : INotifier {
    private readonly INotifier _notifier;
    public NotifierDecorator(INotifier notifier) => _notifier = notifier;

    public void Send(string message) {
        _notifier.Send(message);
        Console.WriteLine("Log: Message sent");
    }
}

// استفاده
INotifier notifier = new NotifierDecorator(new EmailNotifier());
notifier.Send("Hello");
```

✅ 9. Observer Pattern

```
public interface IObserver {
    void Update(string message);
}

public class SmsObserver : IObserver {
    public void Update(string message) => Console.WriteLine("SMS: " + message);
}

public class Notifier {
    private List<IObserver> _observers = new();

    public void Subscribe(IObserver observer) => _observers.Add(observer);
    public void Notify(string message) {
        foreach (var observer in _observers)
            observer.Update(message);
    }
}

// استفاده:
var notifier = new Notifier();
notifier.Subscribe(new SmsObserver());
notifier.Notify("Order placed");
```
